МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение

высшего образования

«СЕВЕРО-КАВКАЗСКИЙ ФЕДЕРАЛЬНЫЙ УНИВЕРСИТЕТ»

Институт перспективной инженерии

Департамент цифровых, робототехнических систем и электроники

Отчет к лабораторной работе № 4

Тема: Структурные паттерны: Компоновщик, Декоратор

Дисциплина: **«Паттерны проектирования»**

Выполнил:

Студент группы ПИЖ-б-о-23-1
направление подготовки
09.03.04 «Программная
инженерия»

Постников Илья Викторович

Проверил:

Доцент ДЦРСИЭ

Альбекова З. М.

Ставрополь 2026

Цель: изучить принцип работы структурных паттернов проектирования «Компоновщик» (Composite) и «Декоратор» (Decorator) и реализовать прикладную задачу «Графический пароль» с использованием данных паттернов.

Задание: необходимо разработать приложение для работы с графическим паролем и реализовать два структурных паттерна из списка: «Компоновщик», «Декоратор».

В соответствии с заданием были выбраны паттерны «Компоновщик» (Composite) и «Декоратор» (Decorator).

Вариант 8 - Графический пароль.

Разработанное приложение позволяет формировать, сохранять и проверять графический пароль, а также демонстрирует работу структурных паттернов: компоновщик используется для группировки шаблонов в наборы, а декоратор - для расширения шаблонов новыми свойствами (избранное, реверс) без изменения их кода.

1. Описание паттерна «Компоновщик»

Паттерн «Компоновщик» (Composite) - это структурный паттерн проектирования, который позволяет объединять объекты в древовидные структуры для представления иерархий «часть - целое». Клиентский код может работать с отдельными объектами и их группами одинаково, не различая их.

Данный паттерн применяется в ситуациях, когда необходимо представить иерархию объектов, и клиент должен единообразно обрабатывать как отдельные элементы, так и их группы.

В разработанном приложении паттерн «Компоновщик» используется для группировки шаблонов графического пароля. В программе определены следующие роли:

- PasswordComponent - абстрактный компонент с методами get_name() и get_sequence();

- PasswordLeaf - простой шаблон (лист), содержащий одну последовательность;

- PasswordComposite - группа шаблонов (композит), содержащая список дочерних компонентов.

При применении композита «Набор: Крест + Квадрат» программа объединяет последовательности детей, удаляет дубликаты и обрезает результат до 9 элементов, после чего применяет полученную последовательность.

Таким образом, клиентский код работает с PasswordComponent, не различая лист и композит.

2. Описание паттерна «Декоратор»

Паттерн «Декоратор» (Decorator) - это структурный паттерн проектирования, который позволяет динамически добавлять объектам новые обязанности, не изменяя их код. Декоратор оборачивает объект и делегирует ему вызовы, добавляя своё поведение до или после.

Данный паттерн применяется в ситуациях, когда нужно расширить функциональность объекта без создания подклассов и без изменения его исходного кода.

В разработанном приложении паттерн «Декоратор» используется для расширения шаблонов графического пароля. В программе определены следующие роли:

- PatternDecorator - базовый декоратор, оборачивающий PasswordComponent;

- FavoriteDecorator - добавляет префикс «★ » к имени шаблона;

- ReversedDecorator - переворачивает последовательность шаблона и добавляет префикс «⇄ ».

Декораторы работают прозрачно: клиентский код вызывает get_name() и get_sequence(), не зная, обёрнут ли компонент. При этом декорированные шаблоны отображаются в общем списке наравне с обычными.

Таким образом, паттерн «Декоратор» позволяет расширять поведение шаблонов без изменения их классов.

3. Описание разработанного приложения

Разработанное приложение предназначено для формирования, сохранения и проверки графического пароля.

Приложение реализовано на языке Python с использованием библиотеки Tkinter.

Основное окно содержит сетку из девяти скруглённых ячеек, пронумерованных от 1 до 9. Пользователь формирует пароль путём последовательного выбора ячеек.

К паролю предъявляются требования:

- длина последовательности от 4 до 9 ячеек;
- одна и та же ячейка не может быть выбрана повторно;
- созданный пароль сохраняется в виде SHA-256 хэша.

Приложение содержит два режима: «Создание пароля» и «Вход».

Также в приложении реализована панель шаблонов, содержащая:

- простые шаблоны: «Крест», «Квадрат», «Зигзаг» (листья);
- составной шаблон «Набор: Крест + Квадрат» (композит);
- декорированные шаблоны «★ Крест» и «⇄ Квадрат» (декораторы).

При клике на карточку шаблона его последовательность применяется к области графического пароля с уведомлением.

4. Диаграмма классов

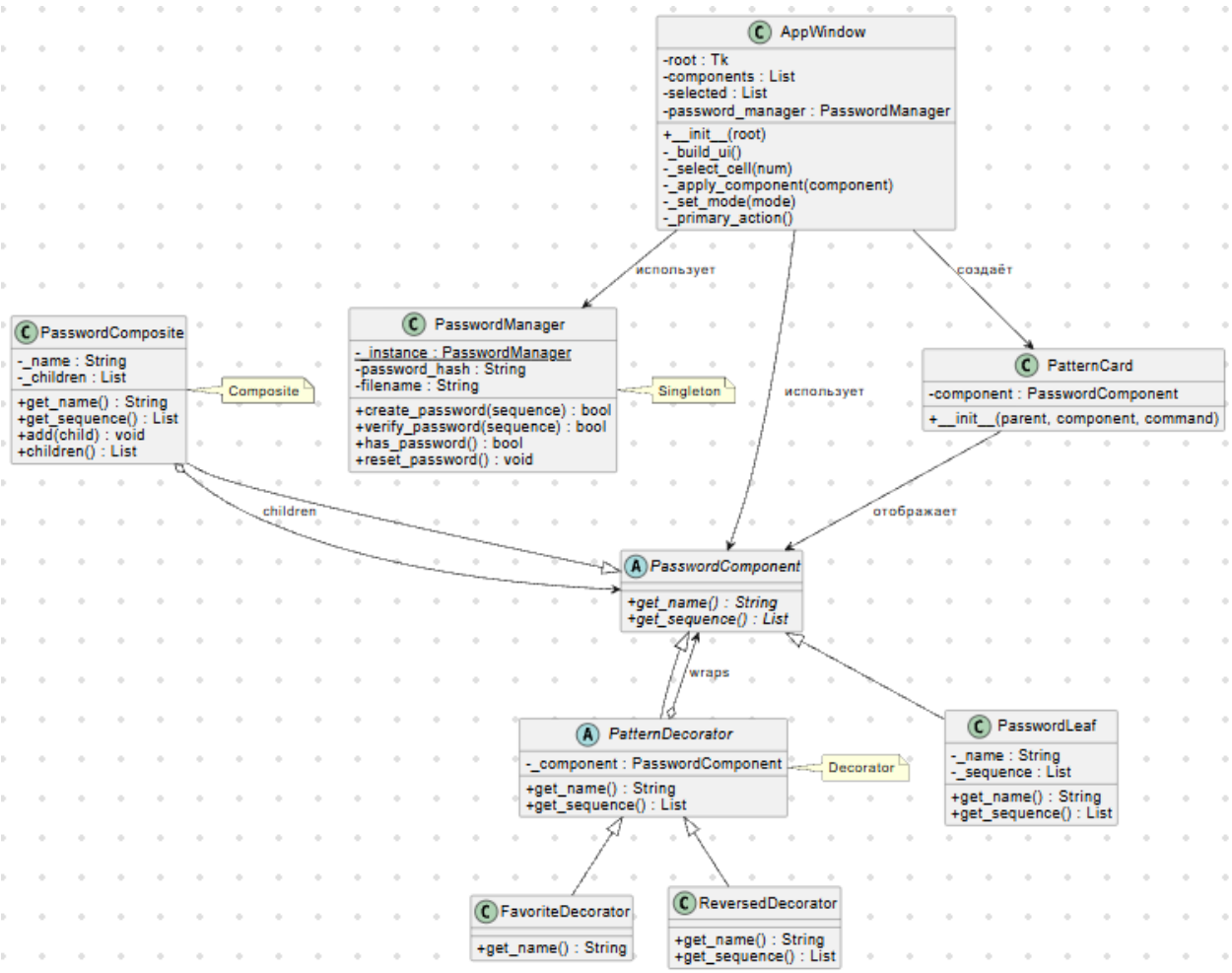


Рисунок 1 - Диаграмма классов приложения «Графический пароль»

5. Реализация паттерна «Компоновщик»

Базовый компонент:

```
class PasswordComponent(ABC):
    @abstractmethod
    def get_name(self) -> str:
        pass


    @abstractmethod
    def get_sequence(self) -> list:
        pass
```

Лист:

```
class PasswordLeaf(PasswordComponent):
    def __init__(self, name, sequence):
```

```
        self._name = name
        self._sequence = sequence

    def get_name(self):
        return self._name

    def get_sequence(self):
        return list(self._sequence)
```

Композит:

```
class PasswordComposite(PasswordComponent):
    def __init__(self, name, children):
        self._name = name
        self._children = list(children)

    def get_name(self):
        return self._name

    def get_sequence(self):
        result = []
        for child in self._children:
            for x in child.get_sequence():
                if x not in result:
                    result.append(x)
                if len(result) >= 9:
                    break
            if len(result) >= 9:
                break
        return result

    def add(self, child):
```

```
        self._children.append(child)

    def children(self):
        return list(self._children)
```

Композит «Набор: Крест + Квадрат» объединяет [2, 5, 8, 4, 6] и [1, 2, 3, 6, 9, 8, 7, 4] в [2, 5, 8, 4, 6, 1, 3, 9, 7].

6. Реализация паттерна «Декоратор»

Базовый декоратор:

```
class PatternDecorator(PasswordComponent):
    def __init__(self, component):
        self._component = component

    def get_name(self):
        return self._component.get_name()

    def get_sequence(self):
        return self._component.get_sequence()
```

Декоратор «Избранное»:

```
class FavoriteDecorator(PatternDecorator):
    def get_name(self):
        return "★ " + self._component.get_name()
```

Декоратор «Реверс»:

python

```
class ReversedDecorator(PatternDecorator):
    def get_name(self):
        return "⇄ " + self._component.get_name()

    def get_sequence(self):
```

```
        return list(reversed(self._component.get_sequence()))
```

В приложении используются:

FavoriteDecorator(PasswordLeaf("Крест", [2, 5, 8, 4, 6])) - отображается как «★ Крест», последовательность та же;

ReversedDecorator(PasswordLeaf("Квадрат", [1, 2, 3, 6, 9, 8, 7, 4])) - отображается как «⇄ Квадрат», последовательность [4, 7, 8, 9, 6, 3, 2, 1].

7. Результаты работы программы

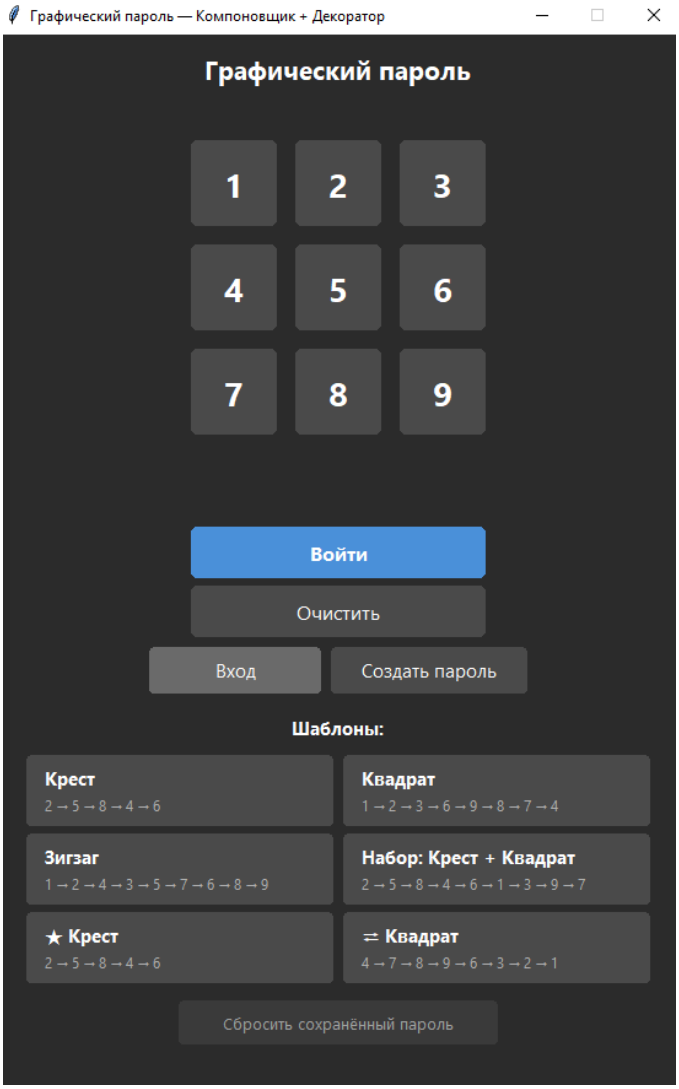


Рисунок 2 - Главное окно приложения

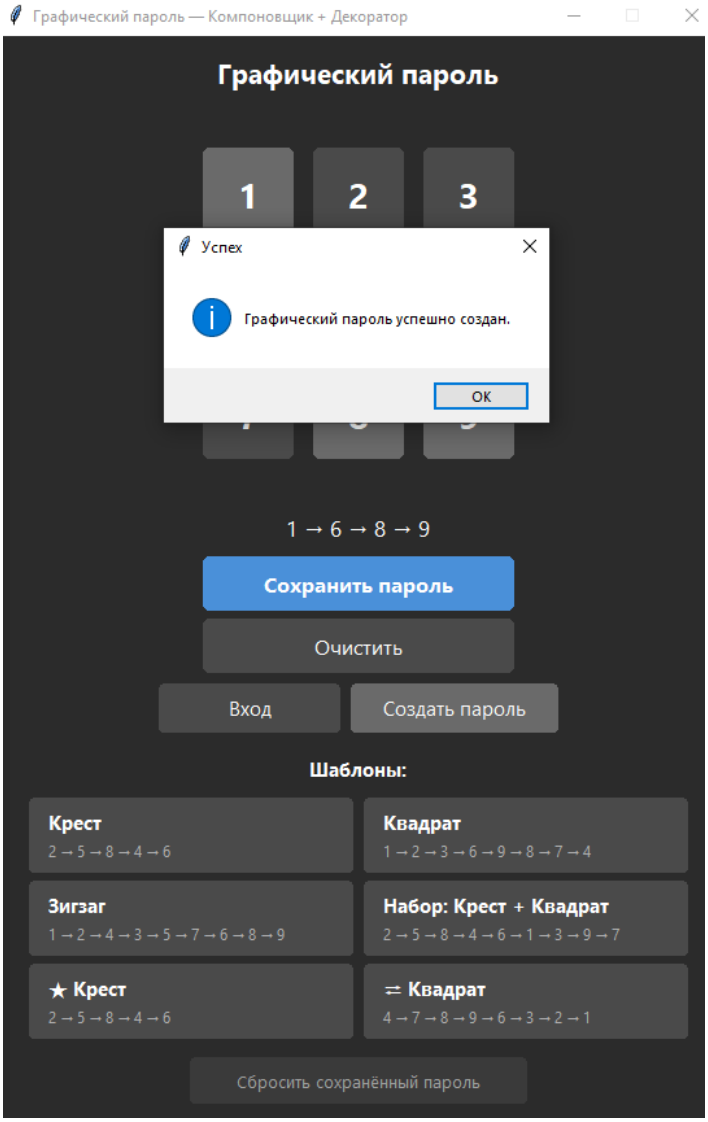


Рисунок 3 - Формирование последовательности

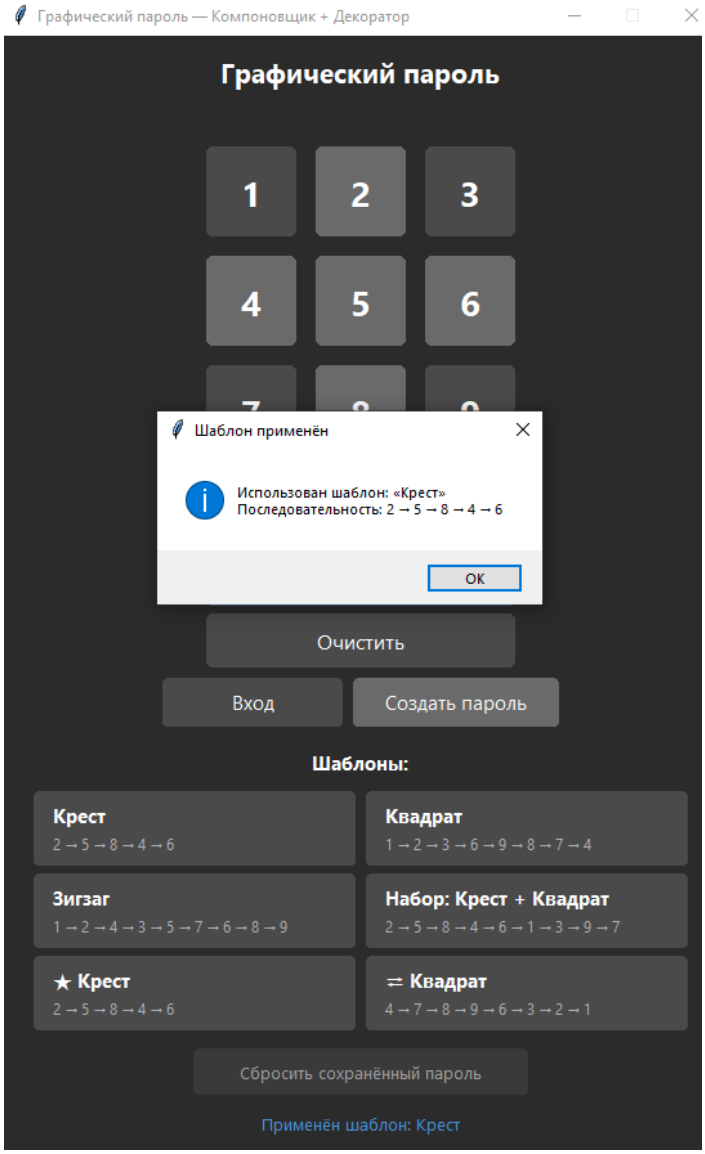


Рисунок 4 - Применение простого шаблона (Крест)

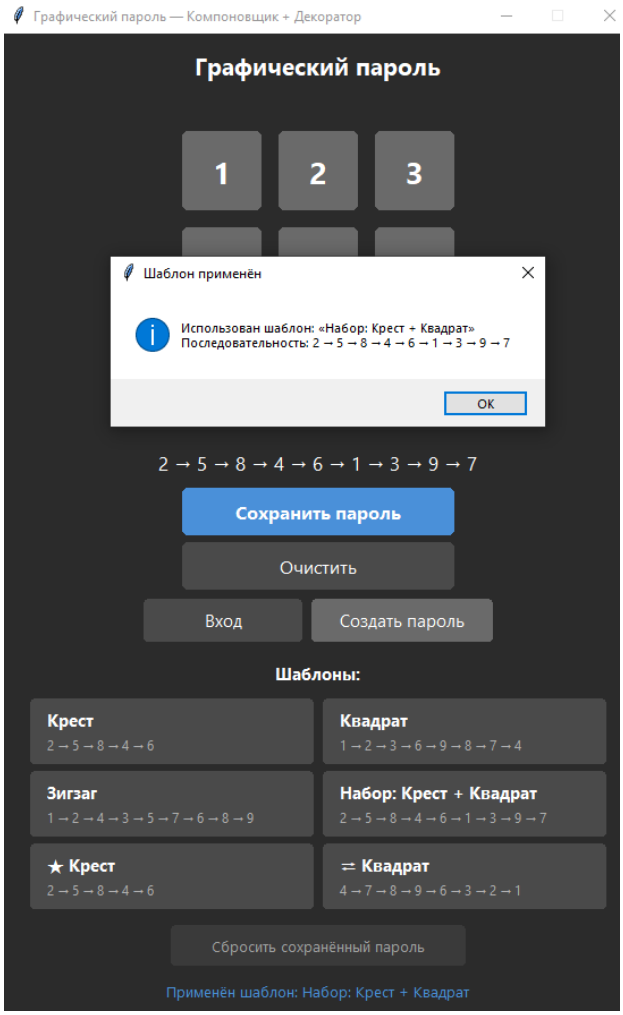


Рисунок 5 - Применение композита «Набор: Крест + Квадрат»

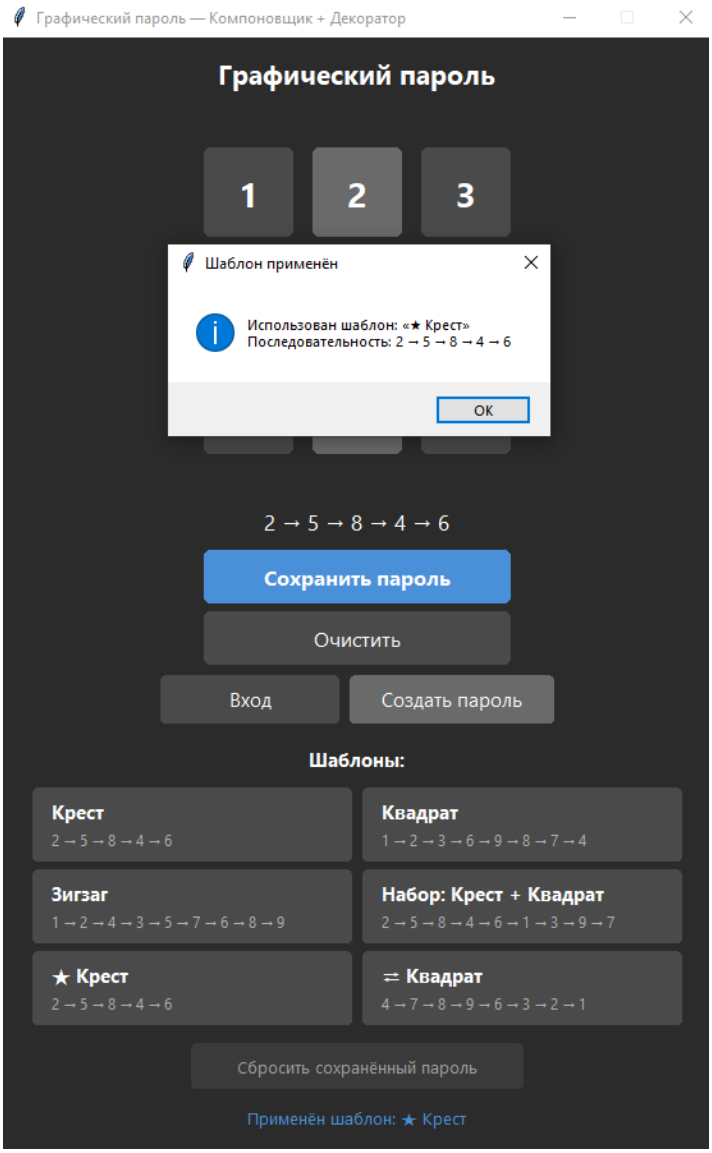


Рисунок 6 - Применение декорированного шаблона «★ Крест»

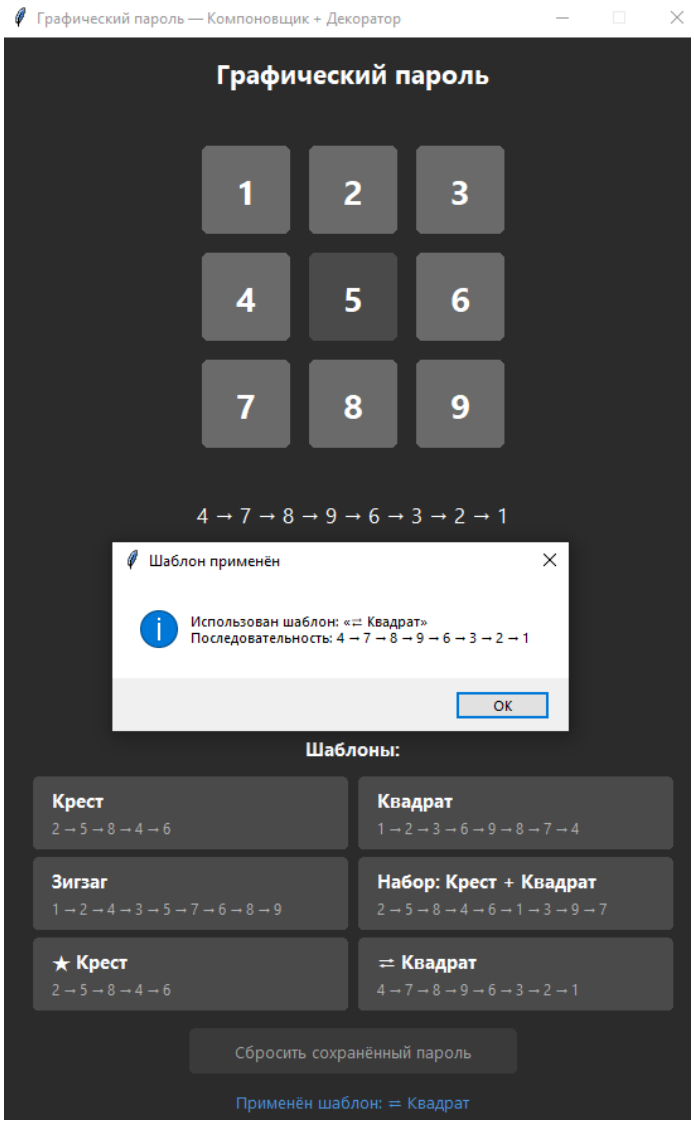


Рисунок 7 - Применение декорированного шаблона «⇄ Квадрат»

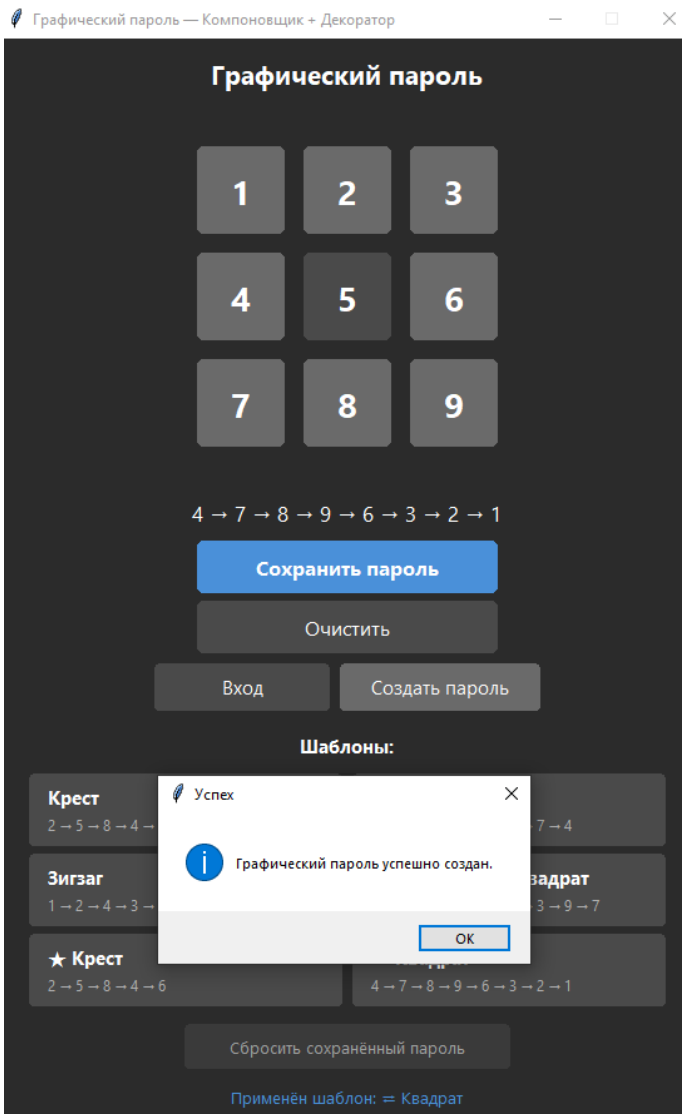


Рисунок 8 - Создание пароля

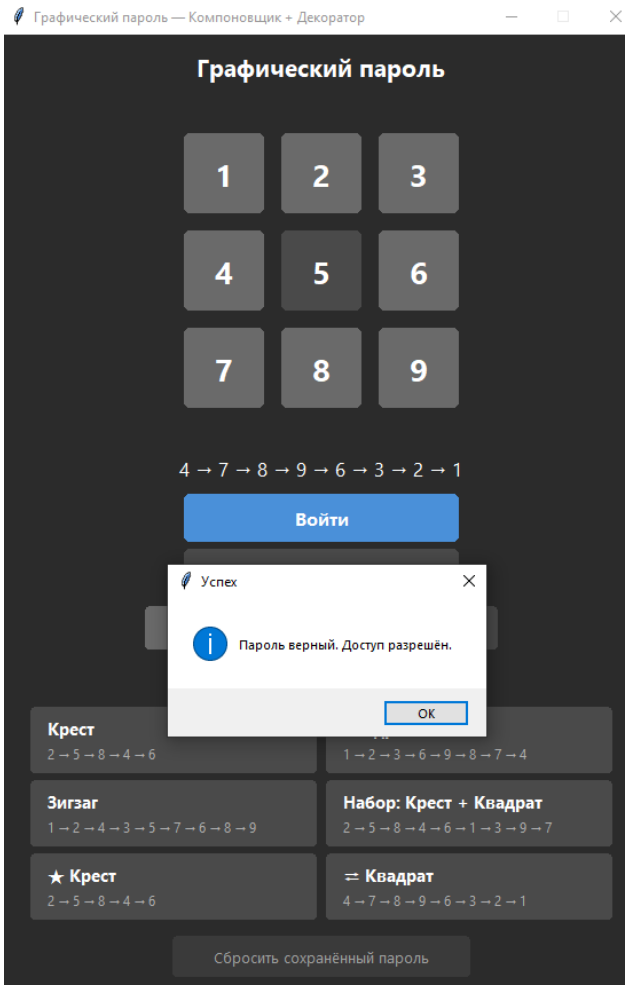


Рисунок 9 - Проверка пароля

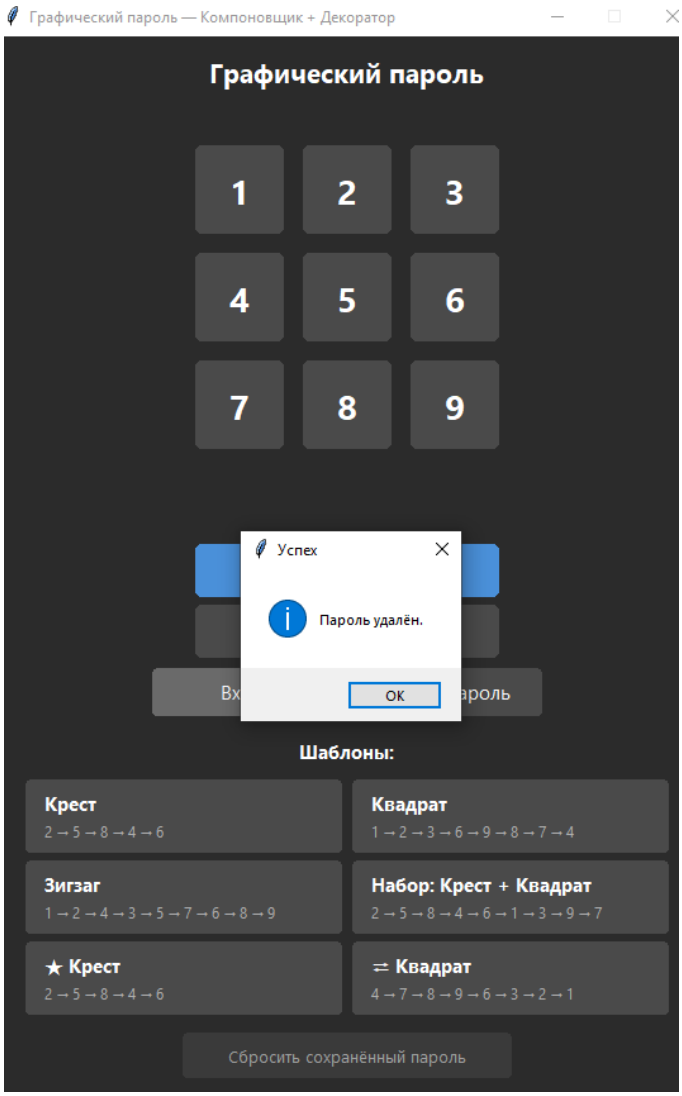


Рисунок 10 - Сброс сохранённого пароля

8. Исходный код

Основные элементы программы:

- PasswordComponent - абстрактный компонент компоновщика;
- PasswordLeaf - лист (простой шаблон);
- PasswordComposite - композит (набор шаблонов);
- PatternDecorator - базовый декоратор;
- FavoriteDecorator - декоратор «избранное»;
- ReversedDecorator - декоратор «реверс»;
- AppWindow - основной класс интерфейса;
- PasswordManager - Singleton для хранения пароля;
- PatternCard - карточка шаблона;

- RoundedButton, rounded_rect() - скруглённые элементы.

Полный исходный код доступен в репозитории GitHub: https://github.com/q1wf3/Patterny_Postnikov

Вывод

В ходе выполнения лабораторной работы было разработано приложение «Графический пароль» на языке Python с использованием библиотеки Tkinter. Были изучены и реализованы два структурных паттерна проектирования: «Компоновщик» (Composite) и «Декоратор» (Decorator). Паттерн «Компоновщик» применён для группировки шаблонов в наборы через классы PasswordComponent, PasswordLeaf и PasswordComposite, что позволяет единообразно обрабатывать как отдельные шаблоны, так и их группы. Паттерн «Декоратор» применён для расширения шаблонов новыми свойствами («избранное» и «реверс») через классы PatternDecorator, FavoriteDecorator и ReversedDecorator без изменения их кода. В результате получена работоспособная программа, демонстрирующая практическое применение структурных паттернов при разработке прикладного программного обеспечения.